



```
In [1]: # yfinance is a popular python Library used for downloading historical data from  
# It simplifies the process of accessing financial data for various securitites,  
  
!pip install yfinance
```

Requirement already satisfied: yfinance in c:\users\chitt\anaconda3\lib\site-packages (0.2.52)

Requirement already satisfied: pandas>=1.3.0 in c:\users\chitt\anaconda3\lib\site-packages (from yfinance) (2.2.3)

Requirement already satisfied: numpy>=1.16.5 in c:\users\chitt\anaconda3\lib\site-packages (from yfinance) (1.26.4)

Requirement already satisfied: requests>=2.31 in c:\users\chitt\anaconda3\lib\site-packages (from yfinance) (2.32.2)

Requirement already satisfied: multitasking>=0.0.7 in c:\users\chitt\anaconda3\lib\site-packages (from yfinance) (0.0.11)

Requirement already satisfied: lxml>=4.9.1 in c:\users\chitt\anaconda3\lib\site-packages (from yfinance) (5.2.1)

Requirement already satisfied: platformdirs>=2.0.0 in c:\users\chitt\anaconda3\lib\site-packages (from yfinance) (3.10.0)

Requirement already satisfied: pytz>=2022.5 in c:\users\chitt\anaconda3\lib\site-packages (from yfinance) (2024.1)

Requirement already satisfied: frozendict>=2.3.4 in c:\users\chitt\anaconda3\lib\site-packages (from yfinance) (2.4.2)

Requirement already satisfied: peewee>=3.16.2 in c:\users\chitt\anaconda3\lib\site-packages (from yfinance) (3.17.8)

Requirement already satisfied: beautifulsoup4>=4.11.1 in c:\users\chitt\anaconda3\lib\site-packages (from yfinance) (4.12.3)

Requirement already satisfied: html5lib>=1.1 in c:\users\chitt\anaconda3\lib\site-packages (from yfinance) (1.1)

Requirement already satisfied: soupsieve>1.2 in c:\users\chitt\anaconda3\lib\site-packages (from beautifulsoup4>=4.11.1->yfinance) (2.5)

Requirement already satisfied: six>=1.9 in c:\users\chitt\anaconda3\lib\site-packages (from html5lib>=1.1->yfinance) (1.16.0)

Requirement already satisfied: webencodings in c:\users\chitt\anaconda3\lib\site-packages (from html5lib>=1.1->yfinance) (0.5.1)

Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\chitt\anaconda3\lib\site-packages (from pandas>=1.3.0->yfinance) (2.9.0.post0)

Requirement already satisfied: tzdata>=2022.7 in c:\users\chitt\anaconda3\lib\site-packages (from pandas>=1.3.0->yfinance) (2023.3)

Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\chitt\anaconda3\lib\site-packages (from requests>=2.31->yfinance) (2.0.4)

Requirement already satisfied: idna<4,>=2.5 in c:\users\chitt\anaconda3\lib\site-packages (from requests>=2.31->yfinance) (2.10)

Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\chitt\anaconda3\lib\site-packages (from requests>=2.31->yfinance) (2.2.2)

Requirement already satisfied: certifi>=2017.4.17 in c:\users\chitt\anaconda3\lib\site-packages (from requests>=2.31->yfinance) (2024.8.30)

[notice] A new release of pip is available: 24.3.1 -> 25.0

[notice] To update, run: python.exe -m pip install --upgrade pip

```
In [2]: import seaborn as sns
import yfinance as yf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor # we use Regressor because it
# from sklearn import metrics
# from sklearn.metrics import accuracy_score, classification_report, Confusion_ma
```

```
In [3]: data = pd.read_csv(r"D:\CAPSTONE PROJECT_DEPLOYMENT\11. CAPSTONE PROJECT_DEPLOYM
data
```

Out[3]:

	Date	Close (BTC)	Volume (BTC)	Close (ETH)	Volume (ETH)	Close (USDT)
0	2017-11-13 00:00:00+00:00	6559.490234	6263249920	316.716003	1041889984	1.009350
1	2017-11-14 00:00:00+00:00	6635.750000	3197110016	337.631012	1069680000	1.006830
2	2017-11-15 00:00:00+00:00	7315.540039	4200880128	333.356995	722665984	1.003180
3	2017-11-16 00:00:00+00:00	7871.689941	5123809792	330.924011	797254016	1.002120
4	2017-11-17 00:00:00+00:00	7708.990234	4651670016	332.394012	621732992	1.001390
...	...	...	...	...	...	...
1822	2022-11-09 00:00:00+00:00	15880.780273	102905151606	1100.169800	38864492427	0.998600
1823	2022-11-10 00:00:00+00:00	17586.771484	83202283721	1299.464600	28581002122	0.997835
1824	2022-11-11 00:00:00+00:00	17034.292969	55871616488	1287.221069	20920539099	0.998218
1825	2022-11-12 00:00:00+00:00	16799.185547	29717699419	1255.268311	10964962767	0.998616
1826	2022-11-13 00:00:00+00:00	16550.126953	25153593344	1228.583984	9734809600	0.998950

1827 rows × 9 columns



In [4]: *#The code fetches historical price data for Bitcoin, Ethereum, Tether, and Binan
#This cleaned data can then be used for further analysis or machine Learning tas*

```

btc = yf.Ticker('BTC-USD')
prices1 = btc.history(period='5y')
prices1.drop(columns=['Open', 'High', 'Low', 'Dividends', 'Stock Splits'], axis

eth = yf.Ticker('ETH-USD')
prices2 = eth.history(period='5y')
prices2.drop(columns=['Open', 'High', 'Low', 'Dividends', 'Stock Splits'], axis

usdt = yf.Ticker('USDT-USD')
prices3 = usdt.history(period='5y')
prices3.drop(columns=['Open', 'High', 'Low', 'Dividends', 'Stock Splits'], axis

bnb = yf.Ticker('BNB-USD')
prices4 = bnb.history(period='5y')
prices4.drop(columns=['Open', 'High', 'Low', 'Dividends', 'Stock Splits'], axis

```

In [5]: *# The parameter lsuffix and rsuffix in the join method are used to add suffixe t
This is necessary to avoid column name conflicts when the two DataFrames have*

```
p1 = prices1.join(prices2, lsuffix = '(BTC)', rsuffix = '(ETH)')
p2 = prices3.join(prices4, lsuffix = '(USDT)', rsuffix = '(BNB)')
data = p1.join(p2, lsuffix = '_', rsuffix = '_')
```

In [6]: data.head()

Out[6]:

	Close(BTC)	Volume(BTC)	Close(ETH)	Volume(ETH)	Close(USDT)	Volu
--	------------	-------------	------------	-------------	-------------	------

Date						
2020-02-03 00:00:00+00:00	9293.521484	30934096509	189.865067	12392875241	0.998913	39
2020-02-04 00:00:00+00:00	9180.962891	29893183716	189.250595	11714191695	1.001745	39
2020-02-05 00:00:00+00:00	9613.423828	35222060874	204.230240	14865434435	1.002838	46
2020-02-06 00:00:00+00:00	9729.801758	37628823716	212.339081	16425589683	0.997216	46
2020-02-07 00:00:00+00:00	9795.943359	34522718159	222.726059	16673443564	0.997751	42



In [7]: data.tail()

Out[7]:

	Close(BTC)	Volume(BTC)	Close(ETH)	Volume(ETH)	Close(USDT)
--	------------	-------------	------------	-------------	-------------

Date					
2025-01-30 00:00:00+00:00	104735.304688	41915744521	3247.780029	19958405782	1.000122
2025-01-31 00:00:00+00:00	102405.023438	45732764360	3298.265137	30128115902	0.999771
2025-02-01 00:00:00+00:00	100655.906250	27757944848	3118.328613	19917507079	0.999843
2025-02-02 00:00:00+00:00	97688.976562	63091816853	2868.692871	42060930305	0.999760
2025-02-03 00:00:00+00:00	95278.476562	102297174016	2595.977539	87337607168	1.000722



In [8]: data.shape

Out[8]: (1828, 8)

In [9]: data.info()

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 1828 entries, 2020-02-03 00:00:00+00:00 to 2025-02-03 00:00:00+00:00
Data columns (total 8 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   Close(BTC)      1828 non-null   float64
 1   Volume(BTC)     1828 non-null   int64
 2   Close(ETH)      1828 non-null   float64
 3   Volume(ETH)     1828 non-null   int64
 4   Close(USDT)     1828 non-null   float64
 5   Volume(USDT)    1828 non-null   int64
 6   Close(BNB)      1828 non-null   float64
 7   Volume(BNB)     1828 non-null   int64
dtypes: float64(4), int64(4)
memory usage: 128.5 KB
```

In [10]: `data.isna().sum()`

```
Out[10]: Close(BTC)      0
Volume(BTC)      0
Close(ETH)       0
Volume(ETH)      0
Close(USDT)      0
Volume(USDT)     0
Close(BNB)       0
Volume(BNB)      0
dtype: int64
```

In [11]: `data.describe()`

```
Out[11]:
```

	Close(BTC)	Volume(BTC)	Close(ETH)	Volume(ETH)	Close(USDT)	Volume(U
count	1828.000000	1.828000e+03	1828.000000	1.828000e+03	1828.000000	1.828000
mean	38001.321976	3.368154e+10	2039.539108	1.702185e+10	1.000365	6.075666
std	22454.875816	2.000601e+10	1133.558373	1.080281e+10	0.002095	3.953772
min	4970.788086	5.331173e+09	110.605873	2.081626e+09	0.974248	9.989859
25%	20327.131348	2.064499e+10	1298.953979	9.650473e+09	0.999911	3.591407
50%	34237.691406	3.021149e+10	1892.687439	1.487863e+10	1.000183	5.159492
75%	53973.750000	4.094040e+10	2971.376770	2.104514e+10	1.000553	7.272640
max	106146.265625	3.509679e+11	4812.087402	8.733761e+10	1.053585	3.006686

Exploratory Data Analysis

In [13]: `print(data.columns)`

```
Index(['Close(BTC)', 'Volume(BTC)', 'Close(ETH)', 'Volume(ETH)', 'Close(USDT)',
      'Volume(USDT)', 'Close(BNB)', 'Volume(BNB)'],
      dtype='object')
```

In [14]: `data['Close(BTC)']`

Out[14]:

Date	Close(BTC)
2020-02-03 00:00:00+00:00	9293.521484
2020-02-04 00:00:00+00:00	9180.962891
2020-02-05 00:00:00+00:00	9613.423828
2020-02-06 00:00:00+00:00	9729.801758
2020-02-07 00:00:00+00:00	9795.943359
...	
2025-01-30 00:00:00+00:00	104735.304688
2025-01-31 00:00:00+00:00	102405.023438
2025-02-01 00:00:00+00:00	100655.906250
2025-02-02 00:00:00+00:00	97688.976562
2025-02-03 00:00:00+00:00	95278.476562

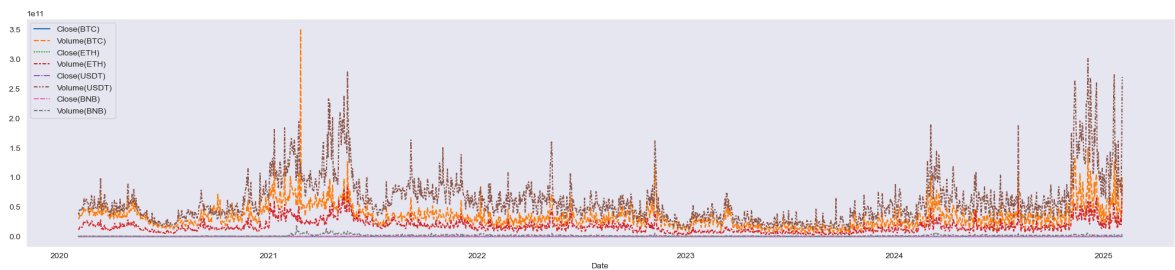
Name: Close(BTC), Length: 1828, dtype: float64

In [15]: `#Visualize the Closing Prices`
`# create a line plot to visualize the closing prices of all four cryptocurrencie`
`plt.figure(figsize=(14, 7))`
`plt.plot(data.index, data['Close(BTC)'], label='Bitcoin (BTC)')`
`plt.plot(data.index, data['Close(ETH)'], label='Ethereum (ETH)')`
`plt.plot(data.index, data['Close(USDT)'], label='Tether (USDT)')`
`plt.plot(data.index, data['Close(BNB)'], label='Binance Coin (BNB)')`
`plt.title('Closing Prices of Cryptocurrencies')`
`plt.xlabel('Date')`
`plt.ylabel('Price (USD)')`
`plt.legend()`
`plt.show()`

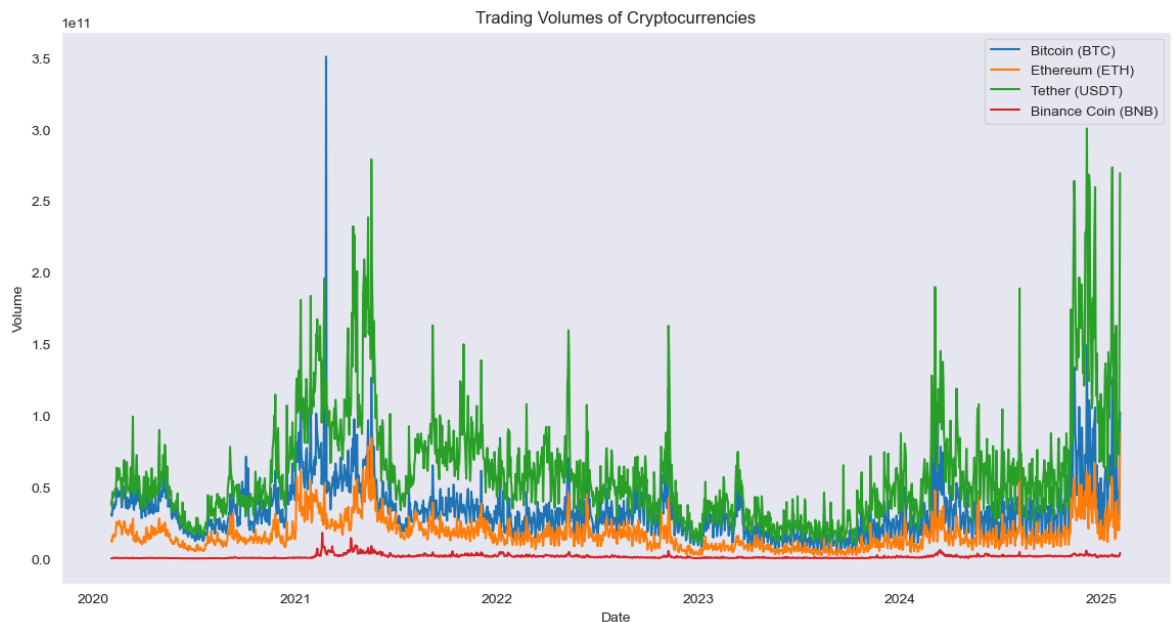


In [16]: `plt.figure(figsize = (25, 5))`
`sns.set_style('dark')`
`sns.lineplot(data=data)`

Out[16]: `<Axes: xlabel='Date'>`

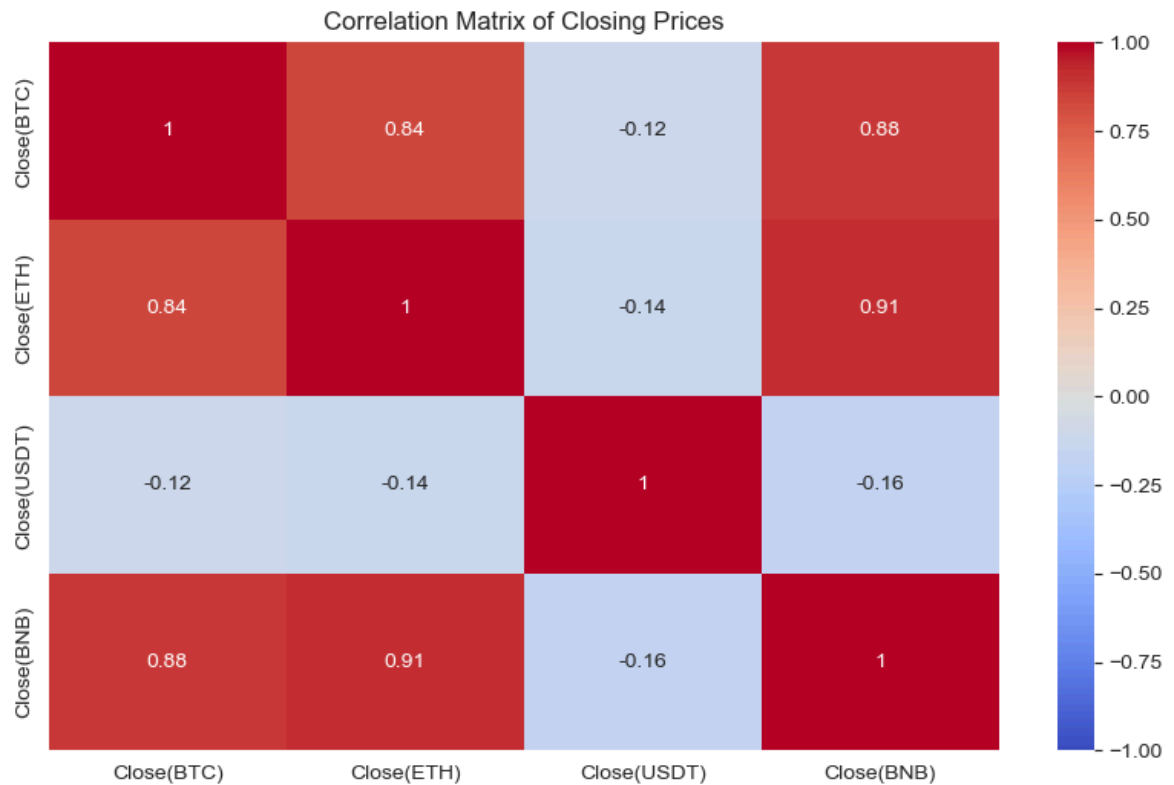


```
In [17]: # Visualize the Trading Volumes
# Let's visualize the trading volumes of all four cryptocurrencies:
plt.figure(figsize=(14, 7))
plt.plot(data.index, data['Volume(BTC)'], label='Bitcoin (BTC)')
plt.plot(data.index, data['Volume(ETH)'], label='Ethereum (ETH)')
plt.plot(data.index, data['Volume(USDT)'], label='Tether (USDT)')
plt.plot(data.index, data['Volume(BNB)'], label='Binance Coin (BNB)')
plt.title('Trading Volumes of Cryptocurrencies')
plt.xlabel('Date')
plt.ylabel('Volume')
plt.legend()
plt.show()
```



```
In [18]: #Correlation Analysis
#We'll analyze the correlation between the closing prices of the cryptocurrencie
# Calculate the correlation matrix
corr_matrix = data[['Close(BTC)', 'Close(ETH)', 'Close(USDT)', 'Close(BNB)']].co

# Plot the heatmap
plt.figure(figsize=(10, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title('Correlation Matrix of Closing Prices')
plt.show()
```

```
In [19]: # Distribution of Closing Prices
#Let's plot the distribution of closing prices for each cryptocurrency:
plt.figure(figsize=(14, 7))

plt.subplot(2, 2, 1)
sns.histplot(data['Close(BTC)'], kde=True, color='blue')
plt.title('Distribution of Bitcoin(BTC) Closing Prices')

plt.subplot(2, 2, 2)
sns.histplot(data['Close(ETH)'], kde=True, color='orange')
plt.title('Distribution of Ethereum (ETH) Closing Prices')

plt.subplot(2, 2, 3)
sns.histplot(data['Close(USDT)'], kde=True, color='green')
plt.title('Distribution of Tether (USDT) Closing Prices')

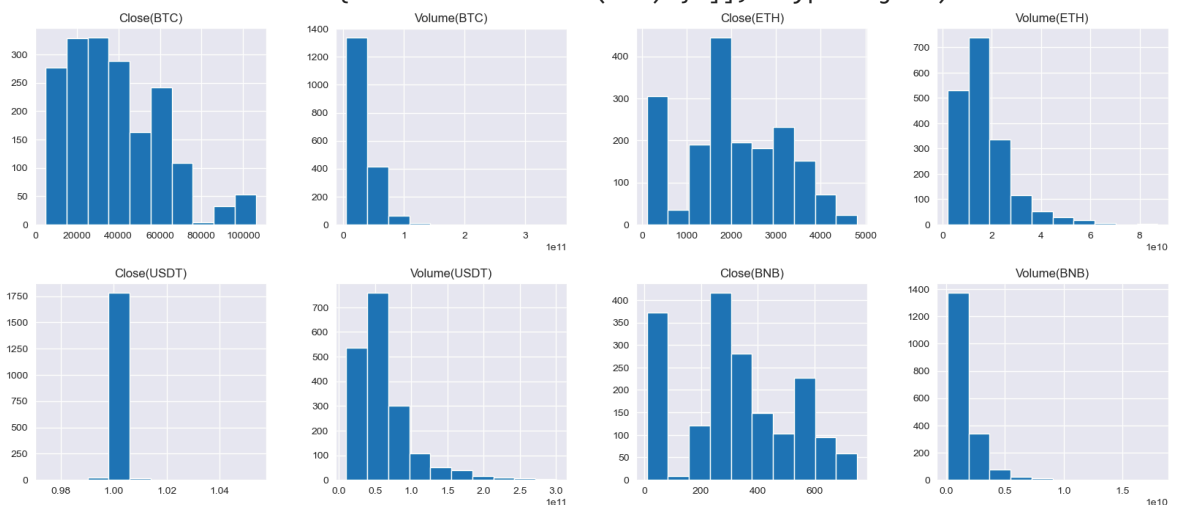
plt.subplot(2, 2, 4)
sns.histplot(data['Close(BNB)'], kde=True, color='red')
plt.title('Distribution of Binance Coin(BNB) Closing Prices')

plt.tight_layout()
plt.show()
```



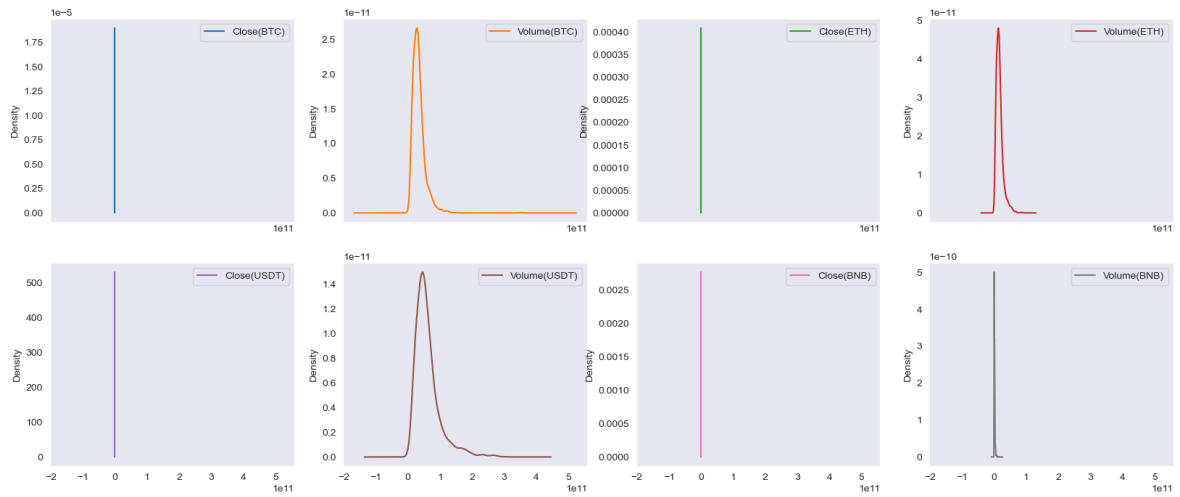

```
In [20]: data.hist(figsize=(20, 8), layout=(2, 4))
```

```
Out[20]: array([[<Axes: title={'center': 'Close(BTC)'>,<Axes: title={'center': 'Volume(BTC)'>,<Axes: title={'center': 'Close(ETH)'>,<Axes: title={'center': 'Volume(ETH)'>],<Axes: title={'center': 'Close(USDT)'>,<Axes: title={'center': 'Volume(USDT)'>,<Axes: title={'center': 'Close(BNB)'>,<Axes: title={'center': 'Volume(BNB)'>]], dtype=object)
```

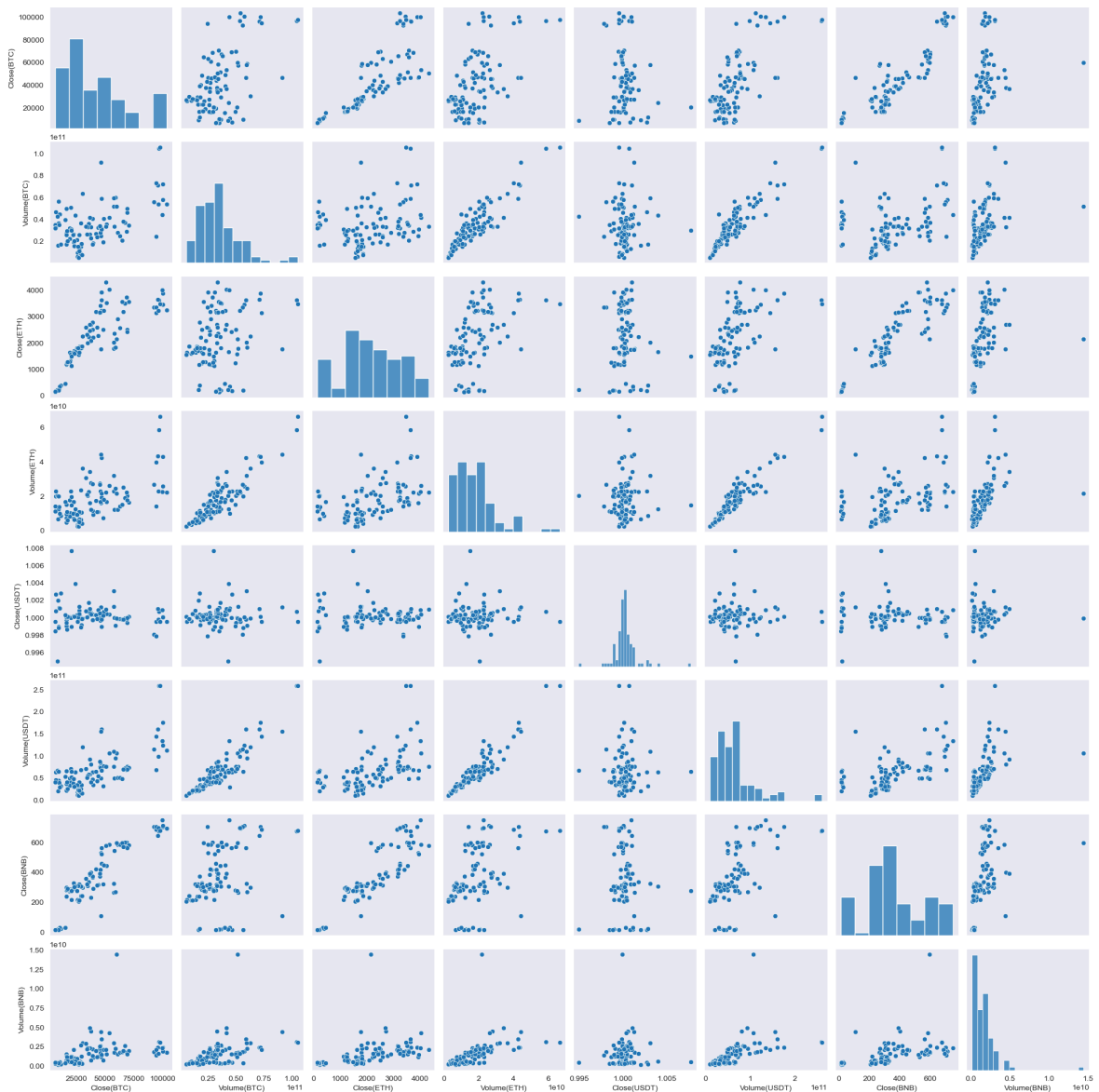


```
In [21]: data.plot(kind = "kde", subplots = True, layout = (2, 4), figsize = (20, 8))
```

```
Out[21]: array([[<Axes: ylabel='Density'>,<Axes: ylabel='Density'>,<Axes: ylabel='Density'>,<Axes: ylabel='Density'>],<Axes: ylabel='Density'>,<Axes: ylabel='Density'>,<Axes: ylabel='Density'>,<Axes: ylabel='Density'>]], dtype=object)
```



```
In [22]: sns.pairplot(data.sample(n=100));
```



Dtaa Pre-processing

```
In [24]: X = data.drop(columns = ['Close(BTC)'], axis = 1)
Y = data.loc[:, 'Close(BTC)']
```

In [25]: `X.head()`

Out[25]:

	Volume(BTC)	Close(ETH)	Volume(ETH)	Close(USDT)	Volume(USDT)	C
Date						
2020-02-03 00:00:00+00:00	30934096509	189.865067	12392875241	0.998913	39849489336	
2020-02-04 00:00:00+00:00	29893183716	189.250595	11714191695	1.001745	36950713371	
2020-02-05 00:00:00+00:00	35222060874	204.230240	14865434435	1.002838	46504757742	
2020-02-06 00:00:00+00:00	37628823716	212.339081	16425589683	0.997216	46549174003	
2020-02-07 00:00:00+00:00	34522718159	222.726059	16673443564	0.997751	42966951015	

In [26]: `X.tail()`

Out[26]:

	Volume(BTC)	Close(ETH)	Volume(ETH)	Close(USDT)	Volume(USDT)
Date					
2025-01-30 00:00:00+00:00	41915744521	3247.780029	19958405782	1.000122	83411405756
2025-01-31 00:00:00+00:00	45732764360	3298.265137	30128115902	0.999771	96233410167
2025-02-01 00:00:00+00:00	27757944848	3118.328613	19917507079	0.999843	72792811129
2025-02-02 00:00:00+00:00	63091816853	2868.692871	42060930305	0.999760	147745588018
2025-02-03 00:00:00+00:00	102297174016	2595.977539	87337607168	1.000722	269471285248

In [27]: `Y.head()`

Out[27]:

```
Date
2020-02-03 00:00:00+00:00    9293.521484
2020-02-04 00:00:00+00:00    9180.962891
2020-02-05 00:00:00+00:00    9613.423828
2020-02-06 00:00:00+00:00    9729.801758
2020-02-07 00:00:00+00:00    9795.943359
Name: Close(BTC), dtype: float64
```

In [28]: `# Split the data into training and testing sets`
`X_train, X_test, Y_train, Y_test = train_test_split(X,Y, test_size = 0.2, random`

In [29]: `# Print the shape of the resulting dataset`
`print(f'X_train shape: {X_train.shape}')`

```
print(f'X_test shape: {X_test.shape}')
print(f'y_train shape: {Y_train.shape}')
print(f'y_test shape: {Y_test.shape}')
```

```
X_train shape: (1462, 7)
X_test shape: (366, 7)
y_train shape: (1462,)
y_test shape: (366,)
```

```
In [30]: # SelectKBest
from sklearn.feature_selection import SelectKBest

fs = SelectKBest(k=4)
X_train = fs.fit_transform(X_train, Y_train)
X_test = fs.transform(X_test)
```

```
C:\Users\chitt\anaconda3\Lib\site-packages\sklearn\feature_selection\_univariate_selection.py:109: RuntimeWarning: invalid value encountered in divide
msw = sswn / float(dfwn)
```

```
In [31]: X_train
```

```
Out[31]: array([[1.00090396e+00, 4.83100617e+10, 2.85084114e+01, 5.31974962e+08],
 [1.00187802e+00, 5.16954245e+10, 2.69931297e+01, 5.12578971e+08],
 [1.00101805e+00, 6.99276614e+10, 4.05069305e+02, 1.43706856e+09],
 ...,
 [1.00026596e+00, 5.14405692e+10, 5.24128540e+02, 1.75798118e+09],
 [1.00064301e+00, 7.75560761e+10, 4.14595032e+02, 1.82613419e+09],
 [1.00018299e+00, 5.58133877e+10, 5.33891357e+02, 1.29457482e+09]])
```

```
In [32]: # MinMaxScaler is a preprocessing method
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
In [33]: # implement of 10 different regression algorithm using scikit-learn.
# Each algorithm is trained and evaluated on a sample dataset
# Import Libraries and Generate Sample Data

from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.svm import SVR
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error, r2_score
```

```
In [34]: #Define Models and Perform Training and Evaluation
models = {
    'Linear Regression': LinearRegression(),
    'Ridge Regression': Ridge(alpha=1.0),
    'Lasso Regression': Lasso(alpha=1.0),
    'ElasticNet Regression': ElasticNet(alpha=1.0, l1_ratio=0.5),
    'Support Vector Regression (SVR)': SVR(kernel='rbf'),
    'Decision Tree Regression': DecisionTreeRegressor(),
    'Random Forest Regression': RandomForestRegressor(n_estimators=100),
    'Gradient Boosting Regression': GradientBoostingRegressor(n_estimators=100),
    'K-Nearest Neighbors Regression': KNeighborsRegressor(n_neighbors=5),
```

```
'Neural Network Regression (MLP)': MLPRegressor(hidden_layer_sizes=(100, 50))
}

# Train and evaluate each model
results = {'Model': [], 'MSE': [], 'R-squared': []}

for name, model in models.items():
    # Train the model
    model.fit(X_train, Y_train)

    # Predict on test set
    Y_pred = model.predict(X_test)

    # Evaluate model
    mse = mean_squared_error(Y_test, Y_pred)
    r2 = r2_score(Y_test, Y_pred)

    # Store results
    results['Model'].append(name)
    results['MSE'].append(mse)
    results['R-squared'].append(r2)

    # Print results
    print(f"----- {name} -----")
    print(f"Mean Squared Error (MSE): {mse}")
    print(f"R-squared: {r2}")
    print()

# Convert results to DataFrame for visualization
results_df = pd.DataFrame(results)
print(results_df)

# Plotting the results
plt.figure(figsize=(12, 6))
plt.barh(results_df['Model'], results_df['R-squared'], color='skyblue')
plt.xlabel('R-squared')
plt.title('R-squared of Different Regression Models')
plt.xlim(-1, 1)
plt.gca().invert_yaxis()
plt.show()
```

----- Linear Regression -----
 Mean Squared Error (MSE): 92408642.61493282
 R-squared: 0.8211378162445313

----- Ridge Regression -----
 Mean Squared Error (MSE): 92457025.29525083
 R-squared: 0.821044168814889

----- Lasso Regression -----
 Mean Squared Error (MSE): 92384764.85704143
 R-squared: 0.8211840330030377

----- ElasticNet Regression -----
 Mean Squared Error (MSE): 411528520.5258781
 R-squared: 0.2034631418011842

----- Support Vector Regression (SVR) -----
 Mean Squared Error (MSE): 512992555.5864274
 R-squared: 0.007073730918974674

----- Decision Tree Regression -----
 Mean Squared Error (MSE): 65694872.018467024
 R-squared: 0.8728438386469681

----- Random Forest Regression -----
 Mean Squared Error (MSE): 37781150.22227781
 R-squared: 0.9268724348467161

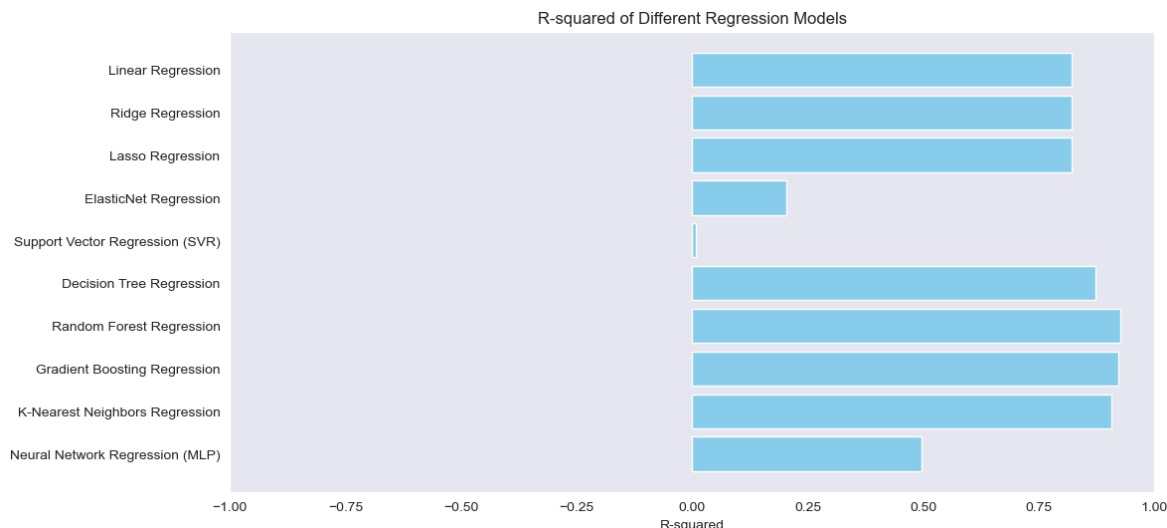
----- Gradient Boosting Regression -----
 Mean Squared Error (MSE): 40021534.16699594
 R-squared: 0.9225360442942353

----- K-Nearest Neighbors Regression -----
 Mean Squared Error (MSE): 47622320.88020421
 R-squared: 0.9078242892969349

C:\Users\chitt\anaconda3\Lib\site-packages\sklearn\neural_network_multilayer_perceptron.py:690: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
 warnings.warn(

----- Neural Network Regression (MLP) -----
 Mean Squared Error (MSE): 260219877.06657526
 R-squared: 0.49632962727680996

	Model	MSE	R-squared
0	Linear Regression	9.240864e+07	0.821138
1	Ridge Regression	9.245703e+07	0.821044
2	Lasso Regression	9.238476e+07	0.821184
3	ElasticNet Regression	4.115285e+08	0.203463
4	Support Vector Regression (SVR)	5.129926e+08	0.007074
5	Decision Tree Regression	6.569487e+07	0.872844
6	Random Forest Regression	3.778115e+07	0.926872
7	Gradient Boosting Regression	4.002153e+07	0.922536
8	K-Nearest Neighbors Regression	4.762232e+07	0.907824
9	Neural Network Regression (MLP)	2.602199e+08	0.496330



Random Forest Regression is a powerful and versatile algorithm suitable for various regression task, offering robust performance and the ability to handle complex data relationship.

Saving the Model

```
In [42]: import pickle
import numpy as np
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, r2_score

# Generate sample data
X, Y = make_regression(n_samples=1000, n_features=10, noise=0.1, random_state=0)

# Scale the features (optional but recommended for some algorithms)
scaler = MinMaxScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Initialize randomforest regressor
model_rf = RandomForestRegressor(n_estimators=100, random_state=0)

# Train the model
model_rf.fit(X_train, Y_train)

# Save the model to a file
filename = 'random_forest_model.pkl'
pickle.dump(model_rf, open(filename, 'wb'))

# Save scaler to a file
with open('scaler.pkl', 'wb') as f:
```



```
pickle.dump(scaler, f)

# Load the model from the file
loaded_model = pickle.load(open(filename, 'rb'))

# Predict using the Loaded model
y_pred = loaded_model.predict(X_test)

# Evaluate the Lodded model
mse = mean_squared_error(Y_test, Y_pred)
r2 = r2_score(Y_test, Y_pred)

print(f"Loaded Random Forest Regression - Mean Squared Error (MSE): {mse}")
print(f"Loaded Random Forest Regression - R-squared: {r2}")
```

Loaded Random Forest Regression - Mean Squared Error (MSE): 260219877.06657526
Loaded Random Forest Regression - R-squared: 0.49632962727680996

In [44]: `import os`
`print(os.getcwd())` *# This shows where your script is running*

C:\Users\chitt

Completed

In []: